1) Recap Tettine

2) Feed Forward Neural Networks (FFNN)

2.1) Recap On Perceptron

The perceptron is a **binary classification algorithm**, that is:

$$f : \mathbb{R}^d \to \{-1, +1\}, \quad x \to t$$

It divides the space into two regions divided by a hyperplane. These regions are called **halfspaces**:

$$H = \left\{x \to sign(w^T x + b) : w \in \mathbb{R}^d, b \in \mathbb{R}\right\}$$

The decision function f is the sign: $y = f(x) = sign(w^T x + b)$

Hyperplane

The hyperplane is the points that satisfy $w^T x = K$. The hyperplane has direction given by w and the distance from the origin is $w^T x = \|w\|\|x\| \cos(\theta)$.

Now add the bias b. The hyperplane is the set of points that satisfy

$$w^T x + b = 0$$

and we define v as the vector from the origin to the closest point of the hyperplane (perpendicular)

$$v = d\frac{w}{\|w\|} = d \cdot u_w$$

The distance can be found by substituting $x = v$:

$$w^T x + b \overset{x=v}{=} w^T d\frac{w}{\|w\|} + b \overset{w^T w = \|w\|^2}{=} d\|w\| + b = 0 \to d = -\frac{b}{\|w\|}$$

From here we get that: **w gives the orientation and b the distance from origin**

for convenience the bias is added as a weight:

$$w_d = \begin{bmatrix} w_1 \\ \dots \\ w_d \end{bmatrix}, \; x_d = \begin{bmatrix} x_1 \\ \dots \\ x_d \end{bmatrix} \to w_{d+1} = \begin{bmatrix} w_1 \\ \dots \\ w_d \\ b \end{bmatrix}, \; x_{d+1} = \begin{bmatrix} x_1 \\ \dots \\ x_d \\ 1 \end{bmatrix}$$

so that $y = sign(w_{d+1}^T x_{d+1}) = sign(w_d^T x_d + b)$.

Training

Given a dataset of labeled points:

$$\mathcal{D} = \{(x_1, t_1), \dots, x_N, t_N\} \quad t_n \in \{-1, +1\}$$

start with $w^{(0)} = \vec{0}$

A correct classification is when $(w^{(i)})^T x_n t_n > 0$.

We must minimize the loss function:

$$\min \mathcal{L}(w, \mathcal{D}) = \min \sum_{n \in \mathcal{D}} l(w, x_n) = \min \sum_{n \in \mathcal{D}} \max(0, -w^T x_n t_n)$$

Weights are updated when a sample is incorrectly classified:

$$w^{(i+1)} = w^{(i)} + x_n t_n$$

At each iteration we are always improving the loss function

Proof:

$$(w^{(i+1)})^T x_n t_n = (w^{(i)} + x_n t_n)^T x_n t_n = (w^{(i)})^T x_n t_n + \|x_n\|^2 > (w^{(i)})^T x_n t_n$$

□

Limitations

This only works for linearly separable data.

A possible solution is feature mapping:

$$\phi(x), \phi : \mathbb{R}^d \to \mathbb{R}^m$$

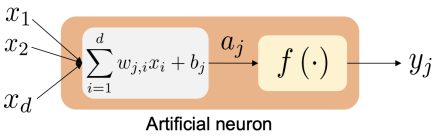
but a good ϕ is hard to find. **Neural networks learn the feature mapping**
There are infinitely many feature mappings, we base them on two insights:

1. ϕ is built as a composition of multiple simpler functions ($\phi = f\ o\ g\ o\ h\ o \ldots$)
2. ϕ has a multi-layered stucture with increasing abstraction

2.2) FFNN

NN are able to learn the feature mapping. Each smaller function (neuron) is based on the perceptron.
Instead of a sign function a differentiable activation function is used.

$$y_j = f(a_j) = f\left(\sum_{i=1}^{d}\right) w_{j,i} x_i + b_j$$



Representation

2.3) Single Layer FFNN

A layer of a FFNN is a **stack of multiple neurons**.
The input vector is fed to a hidden layer (neurons) that have:
-An affine transformation

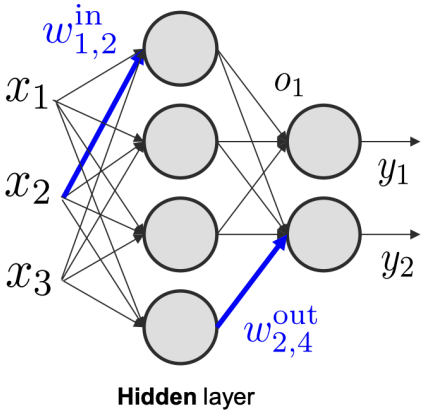
$$a = W^{in} z + b^{in}$$

-A non-linear activation function

$$o = f(a)$$

The output is therefore:

$$y = g(W^{out} o + b^{out})$$



Representation

Theorem 1 (Universal Approximation Capability)**.**

Single-layer FFNN can approximate any function with arbitrary accuracy given a suitable activation function

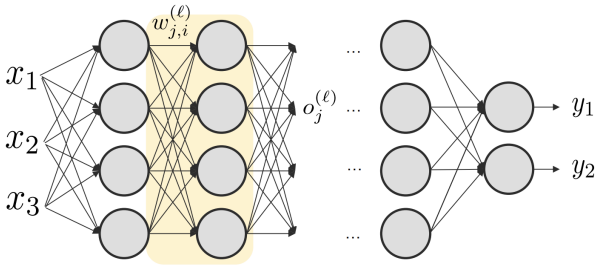
The number of hidden neurons required is exponential in the dimension of the input space d. This means single-layer FFNNs are of limited use in solving practical problems. Multiple layers can be used to solve this problem.

2.4) Multi-Layered FFNN

Notation

Start with some notation:

- $w_{i,j}^{(l)}$: weights that connects neuron i of layer $l-1$ with neuron j of layer l
- $W^{(l)}, b^{(l)}$: parameters of layer l
- $a_j^{(l)}, o_j^{(l)}$: activation and output of neuron j of layer l
- $\mathcal{W}$: set of FFNN parameters (weights and biases)
- $\mathcal{L}(\mathcal{W}, x)$: loss function of FFNN on input sample x
- $\mathcal{D} = \{(x_1, t_1), \ldots, (x_N, t_N)\}$: labeled training data



Repreentation

recall: $w_{\mathbf{j,i}}$ means direction $i \to j$.

Input-Output Relation: Forward Pass

Given an input x, the output is obtained with the forward pass:

- feed x to the first layer: $a^{(1)} = W^{(1)}x + b^{(1)},\ o^{(1)} = f(a^{(1)})$
- Use $o^{(1)}$ as input for the second layer
- Repeat for all layers
- Get output vector as: $a^{(L)} = W^{(L)}o^{(L-1)} + b^{(L)},\ y = g(a^{(L)})$

$a^{(l)}, o^{(l)}\ \forall l \in \{1, \ldots, L\}$ are stored for the back propagation.

Loss Function

The loss function represents the optimization objective of the NN training. It should reflect the learning task.

Typically:

- Negative log-likelihood (NLL) of parameters
- Assume **i.i.d** training samples $\to$ decompose NLL as sum of terms

$$\mathcal{L}(\mathcal{D}, \mathcal{W}) = -\log p(\{t_n\}_{n=1}^N | \{x_n\}_{n=1}^N; \mathcal{W}) = -\sum_{n=1}^N \log p(t_n | x_n; \mathcal{W})$$

Regression (Continuous Labels and Output)

Output Function

for now g is the identity function, therefore $y_n = a_n^{(L)}$
It allows to span the whole target space $\mathbb{R}^n$

Error Function: Mean Squared Error (MSE)

Suppose that label is given by the output + some gaussian noise: $t_n = y_n + \text{noise}$
Then the **per sample likelihood** can be written as a gaussian:

$$\underbrace{p(t_n|x_n;\mathcal{W})}_{\text{objective}} = \mathcal{N}(t_n; y_n, \Sigma)$$

where we suppose y_n is the mean and Σ the covariance.

By setting $\Sigma = I$ so to focus on predicting the mean we can find the following loss function:

$$\begin{aligned} -\log(\mathcal{N}(t_n; y_n, \Sigma)) &= \frac{1}{2}\left[\log\det\Sigma + d\log 2\pi + (y_n - t_n)^T\Sigma^{-1}(y_n - t_n)\right] \\ &\overset{\Sigma=I}{=} \frac{1}{2}\left[d\log 2\pi + (y_n - t_n)^T(y_n - t_n)\right] \\ &\downarrow \\ \mathcal{L}(\mathcal{D}, \mathcal{W}) &\propto \frac{1}{2}\sum_{n=1}^{N}\|y_n - t_n\|^2 \end{aligned}$$

Classification (Discrete Labels, Continuous Output)

Since this is a classification problem, consider the binary classification case:

- 1 output neuron with $t_n \in \{0, 1\}$
- The two classes are called $\mathcal{C}_0, \mathcal{C}_1$

Output Function

Usually the sigmoid function is used:

$$y_n = g(a_n^{(L)}) = \sigma(a_n^{(L)}) = \frac{1}{1 + \exp\left\{-a_n^{(L)}\right\}}$$

This maps $\mathbb{R} \to [0, 1]$
This can be interpreted as the probability of the output being 1 given the input: $p(\mathcal{C}_1|x_n)$
Or alternatively the *complementary class:* $1 - y_n = p(\mathcal{C}_0|x_n)$.

Error Function: Cross-Entropy

Recalling the complementary class, the likelihood can be expressed as:

$$p(t_n|x_n;\mathcal{W}) = p(\mathcal{C}_1|x_n)^{t_n}p(\mathcal{C}_0|x_n)^{1-t_n}$$

and the NLL becomes:

$$\begin{aligned} -\log p(t_n|y_n, \mathcal{W}) &= -t_n \log p(\mathcal{C}_1|x_n) - (1 - t_n)\log p(\mathcal{C}_0|x_n) \\ &= -t_n \log y_n - (1 - t_n)\log(1 - y_n) \\ \mathcal{L}(\mathcal{D}, \mathcal{W}) &= -\sum_{n=1}^{N}\log p(t_n|x_n;\mathcal{W}) \\ &= -\sum_{n=1}^{N} t_n\log(y_n) + (1 - t_n)\log(1 - y_n) \end{aligned}$$

Generalization to multi-class

This can be generalized to multi-class classification problems.

- Let there be Q labels
- Labels are expressed in one-of-Q representation (one-hot encoding)
- Use Q output neurons, each one dedicated to one of the labels

if x_n belongs to a class q, then set $t_{n,q} = 1, t_{n,i} = 0 \forall i \neq q$.
this can be seen as $y_{n,q} = p(t_{n,q} = 1|x_n)$

The error function is the **categorical cross entropy**

$$\mathcal{L}(\mathcal{D}, \mathcal{W}) = -\sum_{n \in N}\sum_{q \in Q} t_{n,q} \log(y_{n,q})$$

Softmax Output

In multi-class, the **output should be a discrete probability distribution overt the Q classes**. Therefore:

- $y_{n,q} \in [0,1],\ \forall q$
- $\sum_{q \in Q} y_{n,q} = 1$

The most common choice is the **softmax function**

$$y_{n,q} = \frac{\exp\left\{a_q^{(L)}\right\}}{\sum_{i \in Q} \exp\left\{a_i^{(L)}\right\}}$$

2.5) FFNN Training

The weight are updated via the **gradient descent** rule

$$(w_{j,i}^{(l)})^{(n+1)} = (w_{i,j}^{(l)})^{(n)} - \eta \nabla_{w_{i,j}^{(l)}} L(W, x)$$

This is calculated via the back propagation algorithm:

1. Execute a forward pass to obtain and store the activations and outputs at each layer
2. For the specific loss and output function, compute

$$\delta_m^{(L)} = \frac{\partial L}{\partial y_m} g'(a_m^{(L)})$$

3. Via the backward pass, compute

$$\delta_j^{(l)} = f'(a_j^{(l)}) \sum_k \delta_k^{(l+1)} w_{k,j}^{(l+1)}$$

4. Finally obtain the gradients for all weights/biases:

$$\frac{\partial L}{\partial w_{j,i}^{(l)}} = \delta_j^{(l)} o_i^{(l-1)} \qquad \frac{\partial L}{\partial b_j^{(l)}} = \delta_j^{(l)}$$

Mathematical Derivation

The aim of backpropagation is to compute the gradient of the loss function with respect to all weights and biases of the network., that is

$$\frac{\partial L(W,x)}{\partial w_{j,i}^{(l)}}, \frac{\partial L(W,x)}{\partial b_j^{(l)}} \quad \forall i,j,l$$

Via the chain rule of derivatives this problem can be recursive and efficient

Forward Pass

The forward pass consists in computing the activations and outputs at each layer of the network given an input vector:
We will store all of the following quantities:

$$o^{(l)}, a^{(l)}\ \forall l, y$$

Chain Rule

Consider the weight $w_{j,i}^{(l)}$ (going from output i at layer $l-1$ to activation j at layer l). The activation will be influenced by the weight (and bias) since

$$a_j^{(l)} = \sum_n w_{j,n}^{(l)} o_n^{(l-1)} + b_j^{(l)}$$

and the two derivatives become:

$$\frac{\partial L}{\partial w_{j,i}} = \frac{\partial L}{\partial a_j^{(l)}} \frac{\partial a_j^{(l)}}{\partial w_{j,i}} = \frac{\partial L}{\partial a_j^{(l)}} o_i^{(l-1)}$$

$$\frac{\partial L}{\partial b_j^{(l)}} = \frac{\partial L}{\partial a_j^{(l)}} \frac{\partial a_j^{(l)}}{\partial b_j^{(l)}} = \frac{\partial L}{\partial a_j^{(l)}}$$

From here define the **error message as**

$$\delta_j^{(l)} \stackrel{\triangle}{=} \frac{\partial L}{\partial a_j^{(l)}}$$

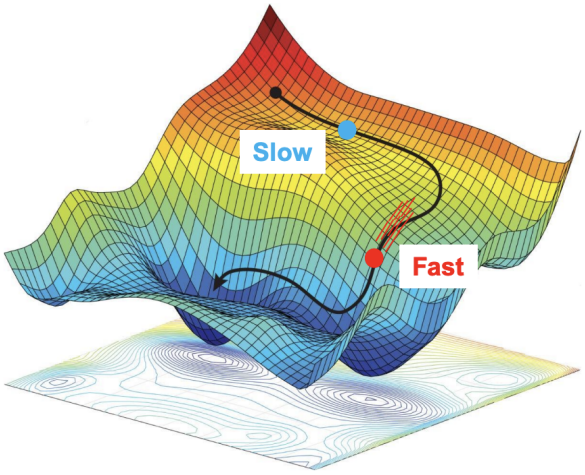
which only depends on layers $> l$.

3) Optimization Methods for Neural Networks

Momentum

Imagine the loss function as a landscape. Imagine a ball rolling on the landscape. The slope makes it descent but constant slope in the same direction gives it momentum. More momentum means more force necessary to divert the ball → This region gives robustness to to momentary changes in the update direction.

How do we give SGD this property?



Example

Smoothing

Find the **cumulative average**

$$h^{(k)} = \sum_{i=1}^{k} \frac{w^{(i)}}{k} \overset{\text{Recursively}}{=} \underbrace{\frac{k-1}{k}}_{\alpha} h^{(k-1)} + \underbrace{\frac{1}{k}}_{\beta} w^{(k)}$$

with $\alpha + \beta = 1$
The recursive form is used to compute average without needing to save all of the weights at every time k
Samples are all of equal weight

We can also use the **exponential average**

$$h^{(k)} = \beta h^{(k-1)} + (1-\beta) w^{(k)}$$

with $\beta \in [0, 1]$ that controls the smoothness of the trajectory.
Samples are not of equal weight: more recent samples are more valuable

---

Return to the ball metaphor

- Position of ball: current parameters of vector $w^{(i)}$
- Velocity of ball $v^{(i)}$ is equal to momentum of unit mass ball
- Gravity force = gradient of the loss function

$$\frac{\eta^{(i)}}{K} \sum_{k=1}^{K} \nabla L(x_k, t_k; w^{(i)})$$

- Update rule:

$$v^{(i+1)} = \alpha v^{(i)} - \frac{\eta^{(i)}}{K} \sum_{k=1}^{K} \nabla L(x_k, t_k; w^{(i)})$$
$$w^{(i+1)} = w^{(i)} + v^{(i+1)}$$

Only the momentum is stored, not the gradients or weights.

Call $g^{(i)} = \frac{1}{K}\sum_{k=1}^{K} \nabla L(x_k, t_k; w^{(i)})$. Now we can see that: (TO FIX THIS FORMULA THERE IS AN ERROR)

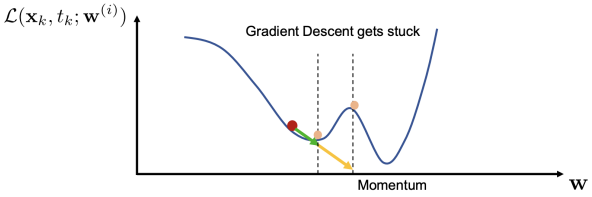
$$
\begin{aligned}
v^{(1)} &= -\eta^{(0)} g^{(0)} \\
v^{(2)} &= \alpha v^{(1)} - \eta^{(1)} g^{(1)} = \alpha(-\eta^{(0)} g^{(0)}) - \eta^{(1)} g^{(1)} \\
&\vdots \\
v^{(k)} &= \alpha^{k-1}(-\eta^{(0)} g^{(0)}) + \alpha^{k-2}() \ldots + \alpha(-\eta^{(k-1)} h^{(k-1)}) - \eta^{(k)} g^{(k)}
\end{aligned}
$$

At first iteration we suppose the ball is at rest, moreover it shows that since $\alpha^n \overset{n\to\infty}{\longrightarrow} 0$ it is clear that more recent weights are more important (similar to exponential smoothing).

Weight updates are less affected by quick changes in direction. Long-term trends in the gradient direction bring acceleration

For gradient based optimization it:

- Accelerates convergence, smooths out wandering behavior
- Makes it easier to escape from local minima



1D Example

Variation: Nesterov Momentum

Look-ahead method: anticipate update to increase responsiveness

This means to consider the gradient in a forward position (possible overshoot scenario) so to have better response:

$$\nabla L(x_k, t_k; w^{(i)} + \alpha v^{(i)})$$

Adaptive Learning Rate Methods

Normalized Gradient Descent

Near stationary points the gradient is close to zero, so the learning gets slower. To solve this we ignore the magnitude of the gradient by normalizing it:

$$w^{(i+1)} = w^{(i)} - \eta \frac{g^{(i)}}{|g^{(i)}|}$$

So the magnitude of the gradient is ignored

$$|w^{(i+1)} - w^{(i)}| = |-\eta \frac{g^{(i)}}{|g^{(i)}|}| = \eta$$

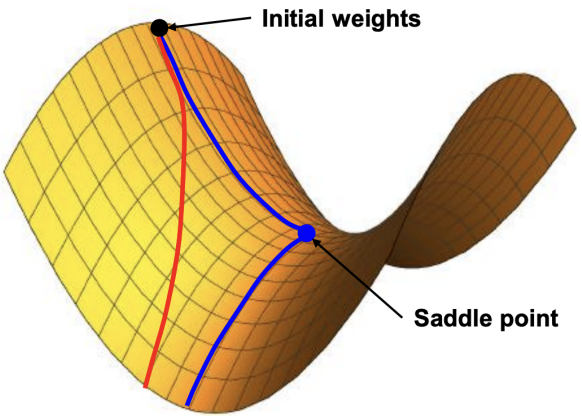
In practice we use

$$w^{(i+1)} = w^{(i)} - \eta \frac{g^{(i)}}{|g^{(i)}| + \epsilon}$$

with ϵ small (10^{-8})

AdaGrad

New idea: component-wise adaptive learning rate (LR)
In fact using the same LR for all components is not recommended as some weights require slower/faster updates than others



Intuition

Recall the gradient on a mini-batch at iteration i $g^{(i)}$
Maintain a running sum of squared gradients **for m-th weight**

$$\alpha_m^{(i)} = \sum_{j=1}^{i} (g_m^{(j)})^2 = (g_m^{(i)})^2 + \alpha_m^{(i-1)}$$

And now compute the weight specific LR

$$\eta_m^{(i)} = \frac{\eta^{(0)}}{\sqrt{\alpha_m^{(i)} + \epsilon}}$$

and thus for the m-th weight:

$$w_m^{(i+1)} = w_m^{(i)} + \eta_m^{(i)} g_m^{(i)}$$

However the sum only grows so the learning rate decreases and might prematurely stop. Use exponentially decaying moving average:

$$r_m^{(i)} = \beta r_m^{(i-1)} + (1-\beta)(g_m^{(i)})^2 = \sum_{k=1}^{i} \beta^{i-k}(1-\beta)(g_m^{(k)})^2$$

this is an IIR filter
The exponents (from i to 1) decay exponentially fast: finite memory approximation by neglecting the terms that $\beta^j < T \to j > \log T / \log \beta$ are after the j-th iteration in the past

Adaptive Moment Estimation (Adam)

Also keep track of the second moment:

$$\begin{aligned} \text{1st Gradient: } \xi_m^{(i)} &= \beta_1 \xi_m^{(i-1)} + (1-\beta_1) g_m^{(i)} \\ \text{2nd Gradient: } \psi_m^{(i)} &= \beta_2 \psi_m^{(i-1)} + (1-\beta_2)(g_m^{(i)})^2 \end{aligned}$$

But this **is not biased** since we set $g_m^{(0)} = 0$.

The bias correction just resorts to

$$\hat{\xi}_m^{(i)} = \frac{\xi_m^{(i)}}{1-\beta_1^i} \qquad \hat{\psi}_m^{(i)} = \frac{\psi_m^{(i)}}{1-\beta_2^i}$$

Proof:
todo

□